

Learning FastAPI: The 20% That Matters

1 The Mental Model

FastAPI is the *server* side of the API picture (the previous document was the client side). You define Python functions, decorate them with a URL path and HTTP method, and FastAPI turns them into web endpoints. Its three superpowers: **automatic data validation** (via Pydantic), **automatic interactive docs**, and **async support**. You write a function; FastAPI handles parsing, validating, and serializing JSON.

2 Setup and Running

```
1 pip install "fastapi[standard]"
2
3 # Run the dev server (auto-reload on file changes)
4 fastapi dev main.py
5
6 # Or the classic way with uvicorn
7 uvicorn main:app --reload
8 # main = the file main.py, app = the FastAPI() object inside it
```

Once running, open <http://127.0.0.1:8000/docs> — free interactive documentation.

3 Your First App

```
1 from fastapi import FastAPI
2
3 app = FastAPI()
4
5 @app.get("/")
6 def read_root():
7     return {"message": "Hello World"}
```

Returning a dict is enough — FastAPI serializes it to JSON automatically.

4 Path Parameters

Parts of the URL itself. The type hint is enforced.

```
1 @app.get("/items/{item_id}")
2 def get_item(item_id: int):          # int -> FastAPI validates & converts
3     return {"item_id": item_id}
4 # GET /items/5          -> {"item_id": 5}
5 # GET /items/abc       -> 422 error automatically (not an int)
```

5 Query Parameters

Function arguments that are NOT in the path become query parameters automatically.

```
1 @app.get("/search")
2 def search(q: str, limit: int = 10, offset: int = 0):
3     return {"q": q, "limit": limit, "offset": offset}
4 # GET /search?q=phones&limit=5
5 # q is required (no default); limit & offset are optional
```

Rule: a parameter with a default is optional; without one it is required.

6 Request Body with Pydantic (The Core Skill)

For POST/PUT, define the expected JSON as a Pydantic BaseModel. FastAPI validates incoming data against it.

```
1 from pydantic import BaseModel
2
3 class Item(BaseModel):
4     name: str
5     price: float
6     in_stock: bool = True          # optional with default
7
8 @app.post("/items")
9 def create_item(item: Item):      # body is parsed into an Item
10     return {"received": item.name, "price": item.price}
```

Send `{"name": "book", "price": 9.99}` and FastAPI gives you a validated `item` object. Wrong types or missing required fields → automatic 422 response with a clear error message. **This is the feature you will use constantly.**

7 The HTTP Method Decorators

```
1 @app.get("/items")          # read
2 @app.post("/items")         # create
3 @app.put("/items/{id}")     # replace
4 @app.patch("/items/{id}")  # partial update
5 @app.delete("/items/{id}") # delete
```

8 Combining Everything

Path param + query param + body in one endpoint:

```
1 @app.put("/items/{item_id}")
2 def update_item(item_id: int, item: Item, notify: bool = False):
3     return {"id": item_id, "data": item, "notify": notify}
```

FastAPI figures out the source of each argument: `item_id` is in the path, `item` is a Pydantic model so it is the body, `notify` is a simple type so it is a query param.

9 Response Models and Status Codes

```
1 from fastapi import status
2
3 class ItemOut(BaseModel):
4     name: str
5     price: float
6     # note: no internal fields -> they get filtered out
7
8 @app.post("/items", response_model=ItemOut, status_code=status.
    HTTP_201_CREATED)
9 def create_item(item: Item):
10     return item # extra fields not in ItemOut are stripped from the
        response
```

`response_model` controls and documents the output shape; `status_code` sets the success code.

10 Error Handling

```
1 from fastapi import HTTPException
2
3 @app.get("/items/{item_id}")
4 def get_item(item_id: int):
5     if item_id not in database:
6         raise HTTPException(status_code=404, detail="Item not found")
7     return database[item_id]
```

Raising `HTTPException` produces a proper JSON error response with the right status code.

11 async vs def

```
1 @app.get("/sync")
2 def sync_endpoint(): # runs in a threadpool, fine for blocking
    work
3     return {"ok": True}
4
5 @app.get("/async")
6 async def async_endpoint(): # use when calling other async I/O (await
    ...)
7     data = await some_async_call()
8     return data
```

Rule of thumb: use `async def` only if you actually `await` something inside. Otherwise plain `def` is safe and FastAPI handles it correctly.

12 Dependency Injection (Key FastAPI Feature)

Reusable logic (auth checks, DB sessions, shared params) declared once and injected.

```
1 from fastapi import Depends
2
```

```

3 def get_db():
4     db = open_connection()
5     try:
6         yield db          # provided to the endpoint
7     finally:
8         db.close()        # cleanup runs after the response
9
10 @app.get("/users")
11 def list_users(db = Depends(get_db)):
12     return db.query_all_users()

```

The same pattern powers authentication: a dependency reads the token, validates it, and returns the user, or raises 401.

13 Pydantic Validation Extras

Pydantic gives you validation for free using standard typing.

```

1 from pydantic import BaseModel, Field, EmailStr
2 from typing import Optional
3
4 class User(BaseModel):
5     name: str = Field(min_length=1, max_length=50)
6     age: int = Field(ge=0, le=120)           # ge=>=, le<=
7     email: EmailStr
8     bio: Optional[str] = None                # optional field

```

14 Automatic Docs (Huge Time-Saver)

Without writing any documentation you get:

- /docs — Swagger UI, interactive: try requests in the browser.
- /redoc — clean read-only reference.
- /openapi.json — the machine-readable schema.

These are generated from your type hints and Pydantic models, which is why typing everything matters.

15 Project Structure (Beyond One File)

Split routes using APIRouter:

```

1 # users.py
2 from fastapi import APIRouter
3 router = APIRouter(prefix="/users", tags=["users"])
4
5 @router.get("/")
6 def list_users():
7     return [...]
8

```

```

9  # main.py
10 from fastapi import FastAPI
11 from users import router as users_router
12
13 app = FastAPI()
14 app.include_router(users_router)

```

16 CORS (Needed for Frontend Calls)

If a browser frontend calls your API, you must allow its origin.

```

1  from fastapi.middleware.cors import CORSMiddleware
2
3  app.add_middleware(
4      CORSMiddleware,
5      allow_origins=["http://localhost:3000"],
6      allow_methods=["*"],
7      allow_headers=["*"],
8  )

```

17 Production Run

```

1  # Multiple workers, no reload, bind to all interfaces
2  uvicorn main:app --host 0.0.0.0 --port 8000 --workers 4

```

This pairs naturally with the Docker document: put that command in your Dockerfile CMD.

18 The Checklist for Any Endpoint

1. Pick the **method** and **path** (`@app.get("/...")`).
2. Decide inputs: path param, query param, or body (Pydantic model)?
3. Define a **response_model** and **status_code** if it matters.
4. Validate with type hints / Pydantic Field.
5. Handle errors with `HTTPException`.
6. Check it in /docs.

19 Practice Task

Build a minimal in-memory “items” API:

1. Define an Item Pydantic model (name, price).
2. POST `/items` to add one to a dict, returning it with a 201 status.
3. GET `/items/{id}` returning the item, or 404 if missing.

4. GET `/items` with an optional `max_price` query param to filter.
5. Open `/docs` and test every endpoint from the browser.